



INSTITUTO FEDERAL DA PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO
DISCIPLINA DE ESTRUTURAS DE DADOS
PROF. VICTOR ANDRÉ PINHO DE OLIVEIRA

Estruturas de Dados

Aula 7 - ED - Filas

Introdução

Olá,

Na semana anterior nós implementamos a Estrutura de Dados Pilha. Fizemos de duas maneiras: uma tomando por base uma Lista Sequencial e outra uma Lista Simplesmente Encadeada.

Hoje veremos uma outra Estrutura: a **Fila**. De semelhante modo, usando as habilidades já adquiridas, veremos que podemos implementá-la usando como base uma Lista Sequencial ou uma Lista Simplesmente Encadeada.

“Facim, facim...”

Filas

Uma **Fila** é uma estrutura semelhante à Lista no sentido de ser uma estrutura que armazena elementos do mesmo tipo. Porém a diferença está na política de acesso aos dados, em que o primeiro elemento inserido deve ser o primeiro a ser retirado. Exatamente por isso, a Estrutura Fila é conhecida pelo termo FIFO (*First In First Out* - Primeiro a Entrar é o Primeiro a Sair). Desse modo, podemos enxergar uma Fila como uma Lista que sempre insere um novo elemento no fim e sempre obtém um elemento do início.

Como uma Fila também é uma variante de uma Lista, apenas com uma política que restringe a forma como os dados podem ser obtidos/acessados, podemos implementá-la usando como base uma Lista Sequencial ou uma Lista Simplesmente Encadeada. Nesta aula, veremos essas duas formas de implementação.

Fila com base em Lista Sequencial

Vamos começar implementando uma Fila usando como base uma Lista Sequencial.

Algumas funções serão aproveitadas, outras foram levemente modificadas e outras foram descartadas. Ao invés da função inserir, agora temos a função **enqueue**, que

insere sempre um elemento no fim da Lista. Ao invés da função obter, temos a função **dequeue** que retorna o elemento do início da Lista já removendo-o. Em outras palavras, **enqueue** enfileira e o **dequeue** desenfileira!

Código

Segue abaixo o código.

```
#include <stdio.h>
#include <stdlib.h>

const unsigned MAX = 10;
int l[MAX], pos = 0;

void enqueue(int elemento);
int dequeue();

int tamanho();
void imprimir();
void apagar();

int main(void) {
    return 0;
}

void enqueue(int elemento){
    if (pos < MAX)
        l[pos++] = elemento;
    else
        printf("Não foi possível inserir %d. Fila cheia.\n", elemento);
}

int dequeue(){
    if (pos) {
        int dado = l[0];
        for (int i = 0 ; i < pos - 1 ; i++)
            l[i] = l[i+1];
        pos--;
        return dado;
    }
    else {
        printf("Não há elementos. Fila vazia.\n");
    }
}
```

```

        exit(1);
    }
}

int tamanho(){
    return pos;
}

void imprimir(){
    printf("F[ ");
    for (int i = 0 ; i < pos ; i++)
        printf("%d ", l[i]);
    printf("]\n");
}

void apagar(){
    pos = 0;
}

```

O código acima mostra uma forma de implementar uma Fila baseada em Lista Sequencial. É preciso de:

- algumas variáveis e uma constante:
 - uma constante **MAX**, para definir o tamanho máximo da Fila
 - um array **l**, para armazenar os elementos da Fila
 - uma variável **pos**¹, para controlar a inserção (enqueue) de elementos na Fila.
- duas funções base:
 - **enqueue**, para enfileirar um novo elemento na Fila
 - **dequeue**, para desenfileirar o primeiro elemento da Fila
- algumas funções auxiliares:
 - **tamanho**, para obter o tamanho da Fila
 - **imprimir**, para exibir toda a Fila
 - **apagar**, para zerar a Fila

Como estamos implementando uma Fila de inteiros, tanto o array **l** quanto as funções foram implementados esperando receber tal tipo.

Adicionalmente, à semelhança do que temos feito, você pode declarar todas as suas funções (assinaturas) acima da main e deixar para implementá-las em qualquer outro lugar abaixo no arquivo.

¹ Algumas implementações trazem o nome fim ao invés de pos. Contudo, resolvi manter o nome da variável como pos para que o aluno notasse as semelhanças com o código da Lista Sequencial.

Lembre-se que, por estarmos implementando uma Fila, que é um tipo de Estrutura de Dados, não podemos manipular o array diretamente, mas somente por meio das funções. As funções ditam as regras de acesso à Estrutura. Assim, basta entendermos como cada função funciona.

Funções base

Começaremos pelas funções base da Estrutura.

Função enqueue

A função enqueue tem o objetivo de inserir um elemento na Fila.

```
void enqueue(int elemento){
    if (pos < MAX)
        l[pos++] = elemento;
    else
        printf("Não foi possível inserir %d. Fila
cheia.\n", elemento);
}
```

A função não retorna nada e recebe o elemento a ser inserido. Primeiramente, verificamos se ainda há espaço para novos elementos. Em caso afirmativo, inserimos na posição pos e incrementamos a variável. Note que foi usado o pós-incremento, o que quer dizer que atribuímos o elemento na posição pos do array l. Só depois é que o incremento é, de fato, aplicado.

Em caso negativo, simplesmente avisamos que não é possível inserir o elemento na Fila por ela já estar cheia.

Função dequeue

A função dequeue tem o objetivo de retornar o primeiro elemento da Fila.

```
int dequeue(){
    if (pos) {
        int dado = l[0];
        for (int i = 0 ; i < pos -1 ; i++)
            l[i] = l[i+1];
        pos--;
        return dado;
    }
    else {
        printf("Não há elementos. Fila vazia.\n");
        exit(1);
    }
}
```

A função retorna o elemento do início da Fila (nesse caso é do tipo inteiro) e não recebe nada. Primeiro verificamos se a Fila não está vazia, avaliando a variável pos. Em seguida, armazenamos em uma variável temporária o primeiro elemento. Isso é necessário para poder “puxar” todos os demais elementos da Fila para preencher o espaço do elemento retirado. Fizemos isso com o for, atribuindo à cada elemento o elemento posterior. Por fim, decrementamos pos e retornamos o dado.

Caso a Fila esteja vazia, situação em que pos é igual a zero, imprimimos o aviso na tela e encerramos o programa. Como a função deve retornar algum elemento, não há como continuar se não houver elemento.

Note que a função dequeue, da forma como implementada acima, possui um custo computacional muito grande, pois, para cada elemento retirado, toda a Fila é percorrida com o objetivo de “puxar” os elementos para cobrir o espaço do elemento retirado. Visto que as operações de enqueue e dequeue são as mais importantes nesse tipo de Estrutura e, logicamente, as mais acessadas, faz-se necessário o uso de uma melhor estratégia de implementação objetivando reduzir o custo computacional no momento da retirada (dequeue).

Uma forma de resolver isso é através da implementação de Filas com Listas Simplesmente Encadeadas, que veremos ainda nesta aula. Outra alternativa, ainda aproveitando a implementação sequencial, seria trabalhar com o array de forma circular, levando-nos à implementação de Filas Circulares. Contudo não veremos esse assunto na aula de hoje.

Funções auxiliares

E agora, veremos as funções auxiliares.

Função tamanho

Retorna o tamanho da Fila.

```
int tamanho(){  
    return pos;  
}
```

Basicamente, a função retorna pos, pois a variável vai sempre coincidir com a quantidade de elementos.

Função imprimir

A função imprimir não recebe nem retorna nada. Apenas imprime o conteúdo da Fila na tela.

```
void imprimir(){  
    printf("F[ ");  
    for (int i = 0 ; i < pos ; i++)  
        printf("%d ", l[i]);
```

```
printf("]\n");  
}
```

Muito semelhante à função imprimir das Listas Sequenciais. Apenas, acrescentamos um F no início da Lista e envolvemos os elementos em colchetes.

Função apagar

A função apagar não recebe nem retorna nada. Tem o objetivo de zerar a Fila.

```
void apagar(){  
    pos = 0;  
}
```

A função está exatamente igual à implementação das Listas Sequenciais. Ao atribuir 0 (zero) à pos estamos dizendo que “não temos elementos na Fila”. Na verdade, como bem sabemos, os elementos estarão no array, mas inacessíveis através da Estrutura. Assim, para todos os efeitos, é como se todos os elementos fossem apagados/removidos da Fila.

Fila com base em Lista Simplesmente Encadeada

Continuando, agora veremos como implementar uma Fila usando como base uma Lista Simplesmente Encadeada.

De semelhante modo, algumas funções serão aproveitadas, outras foram levemente modificadas e outras foram descartadas. Ao invés da função inserir, agora temos a função **enqueue**, que insere sempre um elemento no fim da Lista, e, ao invés da função obter, temos a função **dequeue** que retorna o primeiro elemento da Lista já removendo-o.

Código

Segue abaixo o código.

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct sNODE{  
    int dado;  
    struct sNODE *prox;  
};  
  
struct sNODE *ini = NULL, *fim = NULL;  
  
void enqueue(int dado);  
int dequeue();
```

```

int tamanho();
void imprimir();
void apagar();

int main(void) {
    return 0;
}

void enqueue(int dado){
    struct sNODE *novo = (struct sNODE*) malloc(sizeof(struct
sNODE));
    novo->dado = dado;
    novo->prox = NULL;

    if (!ini)
        ini = fim = novo;
    else{
        fim->prox = novo;
        fim = novo;
    }
}

int dequeue(){
    if (ini) {
        int dado = ini->dado;
        struct sNODE *tmp = ini;

        if (ini == fim)
            ini = fim = NULL;
        else
            ini = ini->prox;

        free(tmp);

        return dado;
    }
    else {
        printf("Não há elementos. Fila vazia.\n");
        exit(1);
    }
}

```

```

}

void apagar(){
    struct sNODE *aux = ini, *ant = NULL;

    while (aux){
        ant = aux;
        aux = aux->prox;
        free(ant);
    }
    ini = fim = NULL;
}

int tamanho(){
    struct sNODE *aux = ini;
    int tam = 0;

    while (aux){
        tam++;
        aux = aux->prox;
    }

    return tam;
}

void imprimir(){
    struct sNODE *aux = ini;

    printf("F[ ");
    while (aux){
        printf("%d ", aux->dado);
        aux = aux->prox;
    }
    printf("]\n");
}

```

O código acima mostra uma forma de implementar uma Fila baseada em Lista Simplesmente Encadeada. É preciso de:

- algumas variáveis e um registro:
 - um registro **sNODE**, contendo um campo **dado** que armazena o elemento e um campo **prox** que aponta para o próximo nó
 - um ponteiro **ini**, que vai sempre apontar para o primeiro nó da Fila

- um ponteiro **fim**, que vai sempre apontar para o último nó da Fila.
- duas funções base:
 - **enqueue**, para enfileirar um novo elemento na Fila
 - **dequeue**, para desenfileirar o primeiro elemento da Fila
- algumas funções auxiliares:
 - **tamanho**, para obter o tamanho da Fila
 - **imprimir**, para exibir toda a Fila
 - **apagar**, para zerar a Fila

Como estamos implementando uma Fila de inteiros, tanto o campo dado do registro sNODE quanto as funções foram implementados esperando receber tal tipo.

Os demais comentários da seção anterior continuam válidos aqui.

Funções base

Começaremos pelas funções base da Estrutura.

Função enqueue

A função enqueue tem o objetivo de inserir um elemento na Fila.

```
void enqueue(int dado){
    struct sNODE *novo = (struct sNODE*) malloc(sizeof(struct
sNODE));
    novo->dado = dado;
    novo->prox = NULL;

    if (!ini)
        ini = fim = novo;
    else{
        fim->prox = novo;
        fim = novo;
    }
}
```

A função não retorna nada e recebe o elemento a ser inserido. Primeiramente alocamos um novo nó e atribuímos o parâmetro dado ao campo dado e NULL ao campo prox, pois o novo nó será inserido no fim.

Em seguida, analisamos o ponteiro ini para identificar se a Fila não está vazia. Caso ini seja NULL (Fila vazia), o novo nó é o único elemento da Fila, então ele é tanto o nó inicial quanto o nó final.

Por outro lado, se já houver elementos na Fila, simplesmente fazemos com que o ponteiro prox do nó fim atual aponte para o novo nó. Só depois fazemos fim apontar para novo.

Função dequeue

A função dequeue tem o objetivo de retornar o primeiro elemento da Fila.

```
int dequeue(){
    if (ini) {
        int dado = ini->dado;
        struct sNODE *tmp = ini;

        if (ini == fim)
            ini = fim = NULL;
        else
            ini = ini->prox;

        free(tmp);

        return dado;
    }
    else {
        printf("Não há elementos. Fila vazia.\n");
        exit(1);
    }
}
```

A função retorna o elemento presente no primeiro nó (nesse caso é do tipo inteiro) e não recebe nada. Primeiro avaliamos o ponteiro ini para averiguar se a Fila não está vazia. Se a Fila não estiver vazia, acessamos o nó ini e salvamos o dado a ser retornado. Em seguida, guardamos o ponteiro para ini em tmp para desalocar o nó mais à frente.

Depois de salvar o ponteiro ini, verificamos se ini e fim são iguais. Se forem iguais, então estamos removendo o único elemento da Fila. Logo devemos ajustar ini e fim para NULL, pois após remover o único elemento da Fila, esta ficará vazia. Caso contrário, apenas avançamos o ponteiro ini.

Por fim, damos um free para liberar o nó apontado por tmp e retornamos dado.

Caso a função seja invocada e a Fila esteja vazia, como não há dado algum para retornar, a função exibe que não há elementos e encerra a aplicação com exit(1) - saída de erro.

Funções auxiliares

E agora, veremos as funções auxiliares.

Função tamanho

Retorna o tamanho da Fila.

```

int tamanho(){
    struct sNODE *aux = ini;
    int tam = 0;

    while (aux){
        tam++;
        aux = aux->prox;
    }

    return tam;
}

```

A função é exatamente igual à implementação na Lista Simplesmente Encadeada.

Função imprimir

A função imprimir não recebe nem retorna nada. Apenas imprime o conteúdo da Fila na tela.

```

void imprimir(){
    struct sNODE *aux = ini;

    printf("F[ ");
    while (aux){
        printf("%d ", aux->dado);
        aux = aux->prox;
    }
    printf("]\n");
}

```

Outra função muito semelhante à função imprimir da Lista Simplesmente Encadeada. A diferença é que estamos incluindo um F antes dos colchetes para indicar que a Estrutura é uma Fila.

Função apagar

A função apagar não recebe nem retorna nada. Tem o objetivo de zerar a Fila.

```

void apagar(){
    struct sNODE *aux = ini, *ant = NULL;

    while (aux){
        ant = aux;
        aux = aux->prox;
        free(ant);
    }
}

```

```
    topo = NULL;  
}
```

Mais uma função exatamente igual à implementação da Lista Simplesmente Encadeada.

Por fim, devemos observar que, do ponto de vista de quem usa a Fila, praticamente não existem diferenças entre a versão com Lista Sequencial e a versão com Lista Simplesmente Encadeada. A única que poderíamos apontar é o limite máximo de elementos imposto pela implementação com Lista Sequencial, o que poderia ser facilmente resolvido se ao invés de uma array estático tivéssemos um ponteiro para um array alocado dinamicamente, realocando o tamanho do array em caso de necessidade. Outra opção seria impor um tamanho máximo na quantidade de elementos na implementação de Fila com Lista Simplesmente Encadeada. São muitas opções, o céu é o limite.

Na aula de hoje implementamos a Estrutura de Dados Fila. As Filas são bastante semelhantes às Pilhas. A diferença reside na política de obtenção de um elemento. Enquanto a Pilha obtém sempre do fim, a Fila obtém sempre do início.

Na próxima aula iremos tratar sobre as Filas Sequenciais Circulares. Tais Estruturas visam, basicamente, reduzir o custo computacional na remoção de elementos no início da Estrutura.

Bons estudos e até a próxima!